Manuel d’utilisation

1. Présentation du projet

Le Robot PFR est un robot mobile développé en 3ème année. Il intègre une Raspberry Pi, un Arduino Mega, une caméra, un LiDAR, des moteurs et des capteurs. Il peut être contrôlé par texte, voix, joystick ou interface web.

1.1 Architecture matérielle

Composant	Modèle	Rôle
Raspberry Pi 3B	Broadcom BCM2837	Cerveau — programme C, serveur Flask, thread web
Arduino Mega	ATmega2560	Contrôle moteurs, capteurs ultrason, Bluetooth
Pi Camera v2	imx219	Flux vidéo MJPEG + détection de balles (cv2)
LiDAR	RPLidar A1	Cartographie 2D de l'environnement
Bluetooth	HC-05	Mode manuel depuis téléphone
Capteurs ultrason	HC-SR04 ×2	Détection obstacles avant et arrière
Moteurs	4× DC + Shield L293D	Déplacement du robot

1.2 Flux de données

> *Utilisateur → Interface web → commande.txt → main.c → action.txt → pilote_robot.py → Arduino → Moteurs*
>
> *Joystick → server_rpi.py → Arduino (direct, sans passer par main.c)*

2. Bibliothèques utilisées

2.1 Bibliothèques C

Bibliothèque	Utilisation
stdio.h / stdlib.h	Entrées/sorties, allocation mémoire
string.h	Manipulation de chaînes de caractères
math.h	Calculs mathématiques (trigonométrie, circularité)
pthread.h	Thread de surveillance de l'interface web (POSIX)
sys/stat.h / unistd.h	Surveillance de fichiers (st_mtime), usleep()

2.2 Bibliothèques Python — Traitement d'image & Robot

Bibliothèque	Utilisation
opencv-python (cv2)	Détection de balles : conversion HSV, masques, contours, cercles englobants, sauvegarde images
numpy	Manipulation de matrices, définition des plages de couleur HSV
math	Calcul de l'angle et de la distance balle/robot
pyserial (serial)	Communication USB avec l'Arduino (/dev/ttyACM0 à 9600 baud)
requests	Récupération du snapshot caméra depuis le serveur Flask (/snapshot)

2.3 Bibliothèques Python — Serveur web

Bibliothèque	Utilisation
flask	Serveur web sur la RPi — routes HTTP, streaming MJPEG
flask-cors	Autoriser les requêtes cross-origin depuis le navigateur
SpeechRecognition	Reconnaissance vocale Google STT depuis l'audio envoyé par le navigateur
subprocess / ffmpeg	Conversion audio WebM → WAV avant reconnaissance vocale
threading	Thread de streaming caméra en parallèle de Flask

2.4 Bibliothèques Python — LiDAR & Cartographie

Bibliothèque	Utilisation
rplidar-roboticslab	Communication avec le RPLidar A1 sur /dev/ttyUSB0
numpy	Traitement des points (angles, distances, séparation murs/obstacles)
opencv-python (cv2)	Morphologie : dilatation et fermeture des contours de la carte
matplotlib	Génération et sauvegarde de la carte PNG (backend Agg — sans écran)

2.5 Bibliothèque Python — Simulation

Bibliothèque	Utilisation
turtle	Simulation graphique du déplacement du robot dans un environnement virtuel

3. Installation

3.1 Accès à la Raspberry Pi

- PuTTY (SSH) : ssh pfr3@172.20.10.8
- VNC Viewer : 172.20.10.8 — accès interface graphique

> Adresse IP sur le réseau hotspot : 172.20.10.8

3.2 Dépendances

```
pip install flask flask-cors pyserial SpeechRecognition requests
opencv-python numpy --break-system-packages
sudo apt install ffmpeg
```

Pour le LiDAR :

```
python3 -m venv venv && source venv/bin/activate
pip install rplidar-roboticslab numpy opencv-python matplotlib
```

3.3 Compilation

```
cd ~/code/PFR_groupe3 && make re
```

4. Démarrage du système

Terminal 1 — Programme principal

```
make re
```

Affiche le menu et lance le thread de surveillance de l'interface web.

Terminal 2 — Serveur web

```
python3 python/server_rpi.py
```

Lance Flask sur le port 5000, démarre le streaming caméra et connecte l'Arduino.

Accès interface web

Ouvrir le fichier **interface_robot.html** directement dans un navigateur.

Ensuite entrer l'adresse IP de la RPi dans le champ CONNEXION RPi (172.20.10.8:5000) et cliquer CONNECTER .

5. Menu terminal

N°	Option	Description
1	Commande clavier	Saisir une commande texte → traiter_commande() → action.txt
2	Commande vocale	Lance assistant_vocal.py (micro RPi) → Google STT → action.txt
3	Simulation	Visualisation graphique Turtle du déplacement
4	Lancer le robot	Lance pilote_robot.py — exécute action.txt sur le robot réel
5	Cartographie LiDAR	Lance scan_lidar.py | display_map.py → génère lidar_map.png
6	Quitter	Ferme le programme

6. Commandes reconnues

Les commandes sont reconnues en français, anglais et espagnol (fichiers config/fr.json, en.json, es.json).

Action	Exemple FR	Exemple EN	action.txt
Avancer	avance 2 m	forward 2 m	advance 2
Reculer	recule 3 m	backward 3 m	retreat 3
Tourner gauche	tourne gauche 90	turn left 90	turn left 90
Tourner droite	tourne droite 45	turn right 45	turn right 45
Trouver balle rouge	trouve la balle rouge	find the red ball	find_ball red
Stop	stop	stop	stop

Valeurs par défaut si non précisées : distance = 1 m, angle = 90°.

7. Interface web

7.1 Voyants d'état

- SERVEUR — connexion Flask active
- ARDUINO — Arduino connectée sur /dev/ttyACM0
- CAMÉRA — flux rpicam-vid actif

7.2 Joystick virtuel

Bouton	Octet	Action	Raccourci clavier
▲	1	Avancer (radar avant actif)	↑
▼	2	Reculer (radar arrière actif)	↓
◂	4	Tourner à gauche	←
▸	5	Tourner à droite	→
STOP	3	Arrêt immédiat	Espace
SPD +	6	Vitesse +20	—
SPD -	7	Vitesse -20	—
📯 BIP	8	Klaxon	—

7.3 Boutons d'action

- 🤖 LANCER LE ROBOT — lance pilote_robot.py (comme option 4 du menu)
- ⚙ MODE ASPIRATEUR — envoie octet '10' → mode autonome d'évitement d'obstacles
- 📡 CARTOGRAPHIE LIDAR — lance le scan et affiche la carte en temps réel
- ⛔ ARRÊT FORCÉ — tue pilote_robot.py, vide action.txt et commande.txt, stop Arduino

7.4 Commande vocale

Cliquer le bouton micro → enregistrement audio → envoi au serveur Flask → conversion ffmpeg (WebM→WAV) → Google STT → résultat dans commande.txt → traité par main.c.

> ⚠ *Nécessite une connexion internet sur la RPi pour Google STT.*

7.5 Flux caméra

Stream Pi Camera v2 en MJPEG (1640×1232, 15 fps) via rpicam-vid. Lors d'une détection de balle, la caméra live se met en pause et les photos de détection s'affichent.

8. Pilotage robot et détection de balles

pilote_robot.py lit data/action.txt et envoie les commandes à l'Arduino. La détection de balles utilise OpenCV et NumPy.

8.1 Traduction des actions

Action dans action.txt	Comportement
advance N	Avance pendant ((N-0.65)/0.77)+1 secondes
retreat N	Recule avec la même formule de durée
turn left/right N	Tourne pendant N×1.15/90 secondes
find_ball couleur	Détecte la balle par vision, calcule l'angle et la distance, se positionne
stop	Arrêt immédiat

8.2 Détection de balle (OpenCV + NumPy)

Le robot récupère un frame depuis le stream caméra via /snapshot (pas de conflit avec rpicam-vid), puis :

- Conversion BGR → HSV avec cv2.cvtColor
- Masque de couleur avec cv2.inRange (plages HSV définies par numpy.array)
- Filtrage des contours : aire > 500 px et circularité ≥ 0.7
- Calcul de l'angle et de la distance depuis la position du cercle dans l'image
- Si non détectée : rotation de 0.5s et nouvelle tentative (max 12 fois)

Photos sauvegardées dans data/photo_brute.jpg et data/photo_detectee.jpg.

9. Arduino

L'Arduino lit les octets sur Serial (USB/RPi) et Serial1 (Bluetooth) simultanément.

9.1 Commandes

Octet	Action	Détails
1	Avancer	Stoppe si obstacle ≤ 50 cm (sonar avant)
2	Reculer	Stoppe si obstacle ≤ 50 cm (sonar arrière)
3	Stop	Arrêt immédiat (RELEASE)
4	Gauche	Rotation sur place
5	Droite	Rotation sur place
6	Vitesse +20	Maximum : 255
7	Vitesse -20	Minimum : 0
8	Klaxon	Bip 100ms
10	Mode aspirateur	Évitement autonome d'obstacles

9.2 Mode aspirateur

Machine à états : avance → détecte obstacle → tourne aléatoirement (400-900ms) → avance. Envoyer '3' pour sortir.

9.3 Mode Bluetooth

- Coupler HC-05 (code : 1234) avec l'application Serial Bluetooth Terminal
- Envoyer les mêmes octets que le tableau ci-dessus

10. Cartographie LiDAR

scan_lidar.py

Connecte le RPLidar A1 sur /dev/ttyUSB0, effectue un scan de 8 secondes et envoie les points (angle, distance) vers stdout.

display_map.py

Reçoit les points, sépare murs et obstacles par analyse statistique, applique une morphologie OpenCV et génère la carte dans data/lidar_map.png avec matplotlib.

> *Backend matplotlib configuré sur 'Agg' (sans écran) pour fonctionner en mode headless sur la RPi.*

Lancer la cartographie

```
. ~/code/PFR_groupe3/venv/bin/activate && python3 python/scan_lidar.py | python3 python/display_map.py
```

Ou directement depuis l'interface web (bouton 📡) ou le menu terminal (option 5).

11. Dépannage

Problème	Cause probable	Solution
Port 5000 occupé	Ancienne instance Flask	sudo fuser -k 5000/tcp
Arduino non détectée	Permissions port	sudo chmod 666 /dev/ttyACM0
Caméra — Device busy	rpicam-vid déjà lancé	sudo pkill -f rpicam-vid
LiDAR — permission refusée	/dev/ttyUSB0 non accessible	sudo usermod -a -G dialout pfr3 + newgrp dialout
STT — Erreur API	Pas d'internet sur la RPi	Vérifier connexion Wi-Fi
Robot ne bouge pas	Serial non lu dans Arduino	Vérifier que Serial ET Serial1 sont lus dans loop()
Balle non détectée	Éclairage insuffisant	Vérifier data/photo_brute.jpg

Commandes utiles

```
hostname -I                          # IP de la RPi
sudo fuser -k 5000/tcp               # Libérer le port Flask
sudo pkill -f rpicam-vid             # Libérer la caméra
> data/action.txt && > data/commande.txt   # Vider les fichiers
```